Demo: https://youtu.be/sKRS7Cmn8y0

GitHub: https://github.com/afischa/UltimateTicTacToe

(presentation slides on GitHub)

Ultimate Tic Tac Toe AI

Group 2: Elliott Tai, Asa Fischer, Riley Marek

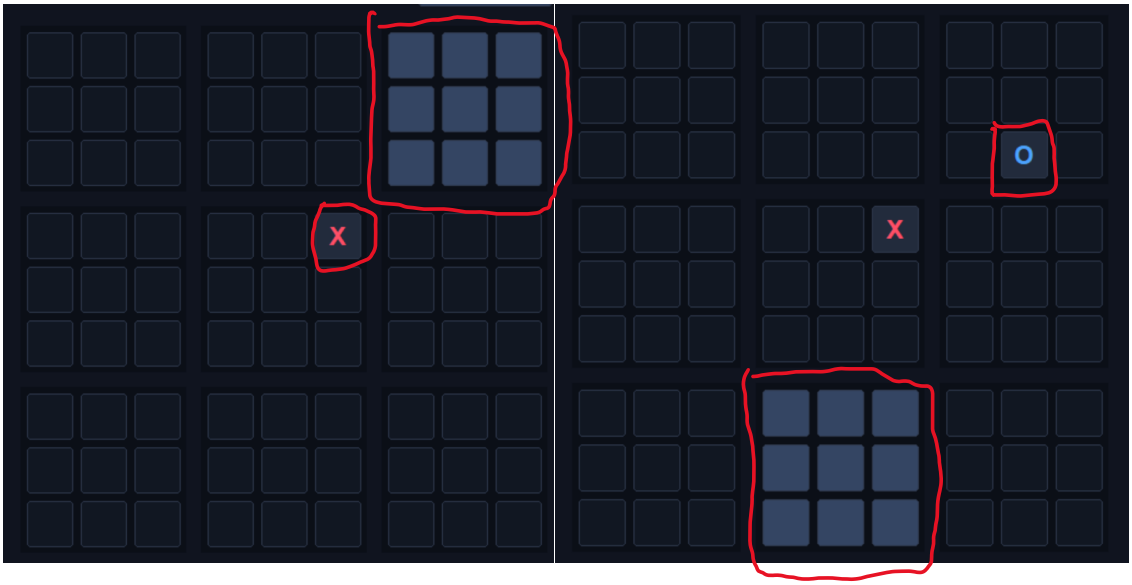
Goal

We wanted to make an AI that could play Ultimate Tic Tac Toe using the minimax tree method discussed in class.

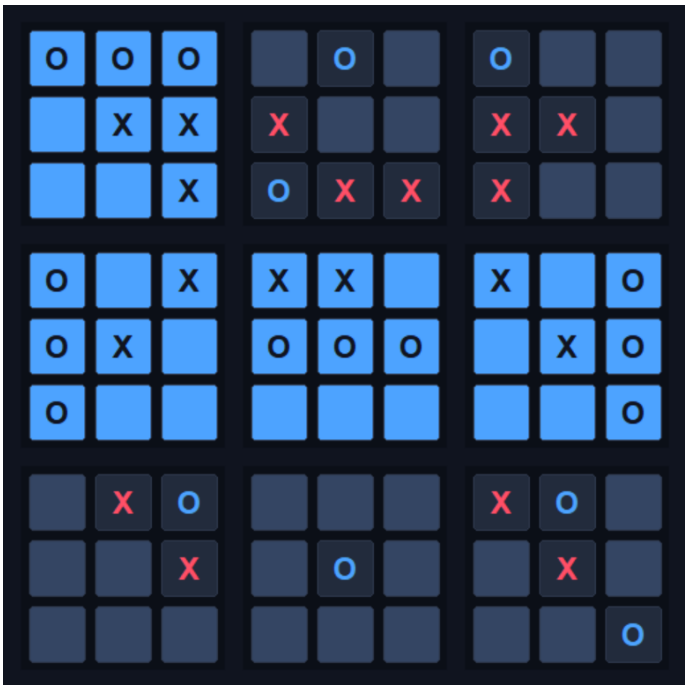
Ultimate Tic Tac Toe

In Ultimate Tic Tac Toe, 9 standard Tic Tac Toe boards are combined in a square pattern to make a global board. To claim a local bord a player must get 3 in a row just like in normal Tic Tac Toe. To win the game a player must claim three local boards in a row.

Special constraint: The location in which a player places their mark within a local board determines the local board their opponent must play in (see example).



Player X placed their mark in the top right corner of a local board so O must now play in the top left local board. Then O played bottom middle so X must now play bottom middle local board.

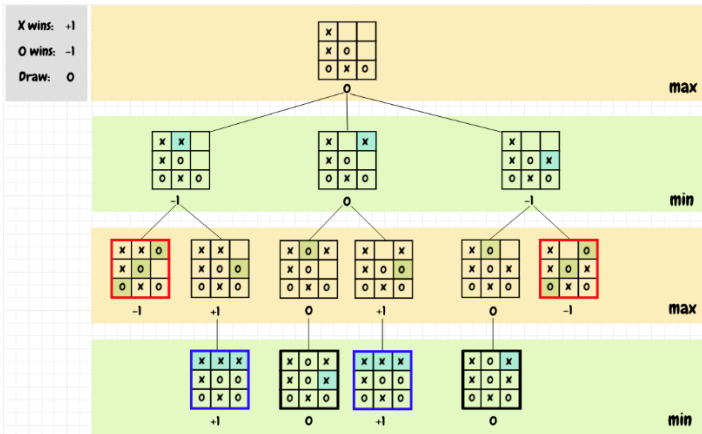


To claim a local board a player must get 3 of their marks in a row just like in normal tic tac toe. If a player gets 3 local boards in a row they win the game. In the image above player O claimed the 3 local boards in the middle row winning the game.

Minimax Tree

In class we learned about the minimax tree method to solve standard Tic Tac Toe. This is where you create a tree with your current state as the root. You make a child for every possible move you could make. So, you would have a child node for every state possible in one move. Then you would create child nodes for each of these nodes for every possible move the opponent could make. You continue to do this until the leaf nodes of the tree are completed games. So, you have all possible outcomes from your current state.

Then you assign a value to each of the leaves, 1 if you win, 0 to a draw, and -1 if the opponent wins. Then you make your way back up the tree assuming that your opponent will pick the minimum value of its children and you will pick max value of children. Once at the root you look at your possible next moves and pick one with the highest value. This is the best move you can make.



https://nestedsoftware.com/2019/06/15/tic-tac-toe-with-the-minimax-algorithm-5988.123625.html

Applied to Ultimate Tic Tac Toe

This same strategy can be applied to Ultimate Tic Tac Toe where we make a tree of all possible paths of both players' decisions. The problem is that a tree like this would be too big to be practical. Even a short game of Ultimate Tic Tac Toe is about 40 moves, assuming there are about 7 possible choices for a player to make, the last layer of the tree alone would be on the order of 7^{40}nodes. For context this is more nodes than there are stars in the observable universe. It is not possible to make a tree this large with moder hardware.

Reduces depth

Instead, we only make a tree a few layers deep at each decision. But now we don't reach a game complete state at the leaves of the tree, so how do we do minimax if there are no values at the leaf nodes? The solution is to make a heuristic that estimates positional advantage, returning a high value for good positions for the AI and low values if the opponent is in a good position.

Our Heuristic

The component that has the highest impact on the heuristic score is a player claiming local boards. A move that results in 3 local boards in a row is awarded a massive score as this is winning the game. Getting 2 local boards in a row has extreme value and claiming a local board that blocks an opponent is also awarded many points. Claiming a local board with none of the previously listed interactions is still highly weighted.

Then the next level of importance has to do with where you are forcing your opponent to play next (does playing this move allow the opponent to make a high impact move?).

The aspect with the lowest impact on the heuristic, is strength of a position within a local board (valuing 2 in a rows or blocking opponent withing a local board). Additionally claiming the center local board and the center space within a local board is assigned more value than the other spaces.

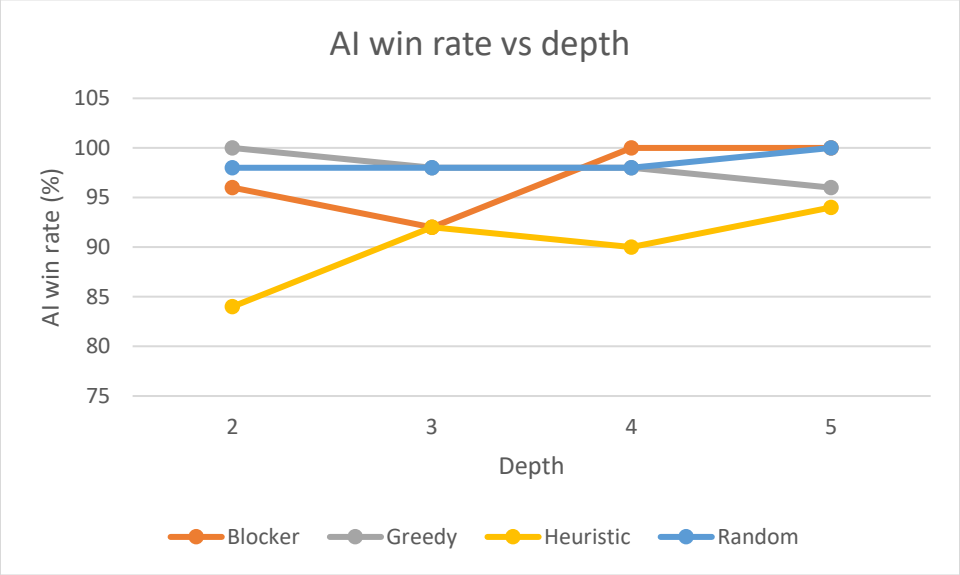
Alpha-beta pruning

In addition to keeping the depth of the tree short we can use alpha-beta pruning to speed up the AI's calculation. At a high level this is a way to avoid evaluating branches of the tree that cannot possibly affect the final decision. For example, if you are looking for a max and have already found a +5, and you are exploring another branch where an min has forced you to a +3 you can stop looking at this branch. No matter what you won't use the result of that branch as it is guaranteed to have a smaller value. This can save a lot of time as even a tree depth 5 can have tens of thousands of nodes and ignoring a portion of them will cause a significant seed-up.

Testing

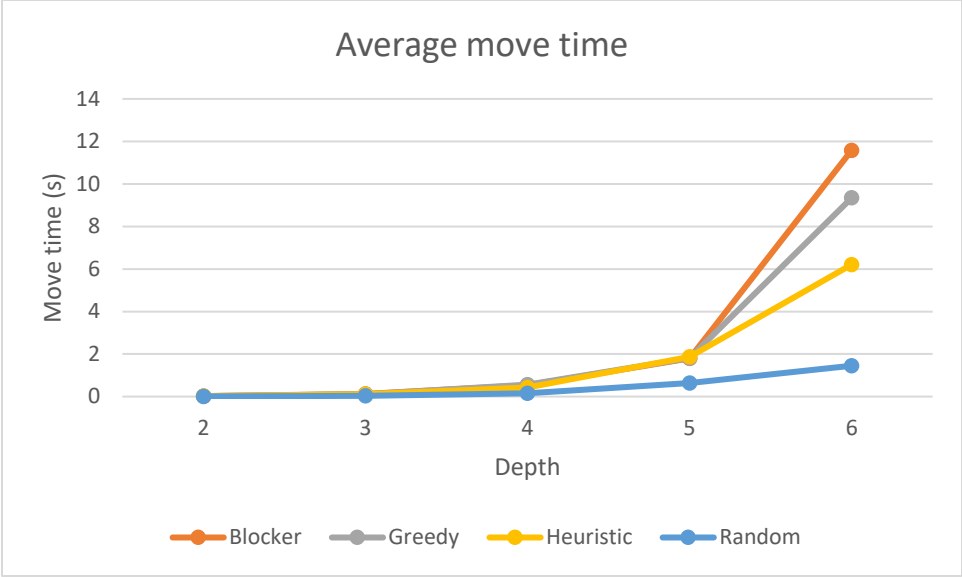
We tested our AI against a variety of opponents. The first of which simply makes random moves. One that is greedy and just tries to win local boards. One that tries to block its opponents from making 3 in a row. And an opponent with a heuristic but no minimax tree.

We measured our AI's performance and decision times against the opponents for various tree depths. It played 100 games against each opponent for each depth value.



You can see that win rate increases as depth is increased. There is the biggest increase against the heuristic and blocking opponent. This is because at low depth our AI is essentially just a heuristic function but as depth is increased our AI gains an advantage over the heuristic only opponents.

Against the greedy and random opponent our heuristic function with little help from minimax is able to beat them quite consistently. So adding depth to the tree has little affect as our AI is already win at nearly 100%.



Average Decision time grows exponentially as depth increases. This makes sense as the tree grows exponentially in size as depth increases.

Depth 4 seems to be the sweat spot as this is where we start to encounter diminishing returns in our win rate and is the depth right before decision times start to explode. When playing against depth 4 you do not get bored waiting for the AI to make it's move. With a depth of 5 or greater there is a noticeable wait time while the AI is thinking.

Result tables

win rate				
Depth	Blocker	Greedy	Heuristic	Random
2	96	100	84	98
3	92	98	92	98
4	100	98	90	98
5	100	96	94	100

ave time per step				
Depth	Blocker	Greedy	Heuristic	Random
2	0.0217	0.0258	0.0197	0.0063
3	0.1198	0.1164	0.1036	0.0378
4	0.5307	0.5656	0.4329	0.1569
5	1.7948	1.806	1.869	0.6327
6	11.5831	9.3437	6.2082	1.4505